Logic Networks Project

Academic Year 2023/2024

POLITECNICO
MILANO 1863

Lucrezia Maestrello

CONTENTS

1 Introduction **3**

1.1 PROJECT AIMS AND GENERAL DESCRIPTION 3

1.2 COMPONENT INTERFACE 4

2 Architecture **5**

2.1 DESIGN CHOICES 5

2.2 DATA PATH 5

2.3 CREDIBILITY_COUNTER 6

2.4 ADD_COUNTER 6

2.5 K_COUNTER 7

2.6 FINITE STATE MACHINE (FSM) 8

3 Experimental Results **10**

3.1 TEST BENCH 10

3.2 SUMMARY REPORT 13

4 Conclusions **14**

1 Introduction

1.1 Project Aims and General Description

The aim of the project is to implement a hardware module (described in VHDL) that interfaces with a memory and, by reading a string of length K from it (bit by bit), modifies it according to the specified requirements.

Specifically, the K-words (whose values range from 0 to 255) are stored starting from a specified address (ADD) every 2 bytes. The missing byte between two values represents the 'credibility', which can take a value between 31 (if the previously read data is not 0) and 0 (whenever a zero is read as a word, the credibility is decremented by 1 from its previous value). Furthermore, all words read as 0 are replaced with the last valid value in the sequence (non-zero).

The module to be implemented has three primary inputs and a single primary output: the inputs consist of a 1-bit signal (START), a 16-bit signal (ADD) (which represents the memory address from which to start reading) and a 10-bit signal (K). The primary output, on the other hand, is a 1-bit signal (DONE).

The START signal indicates the start of the computation and will remain high until DONE is also set to 1. Once processing is complete, the DONE signal will assume a logic 1 value and must remain so until START returns to 0. Only then will DONE return to 0 and a new start signal be provided.

The CLOCK and RESET signals are common to the entire system. All signals are synchronous except for the RESET signal, which is asynchronous. This signal may be set to 1 at any point during processing, regardless of the value of START or the CLOCK signal. The component must operate with a CLOCK period of at least 20 ns.

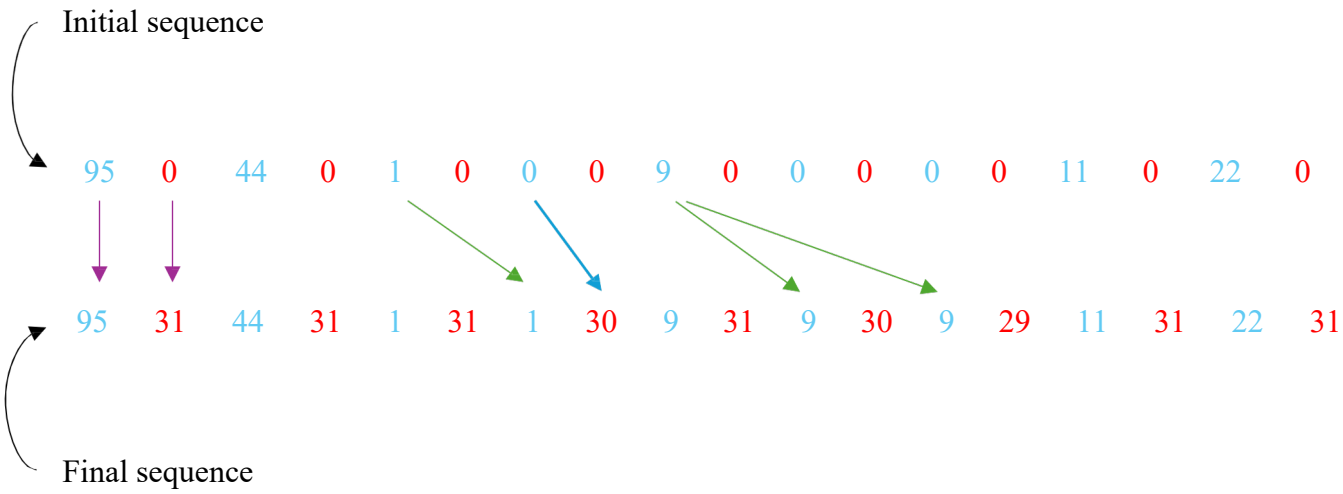

Figure 1: Example of component operation

1.2 Component Interface

The interface of the component to be developed is as follows:

```
entity project_reti_logiche is
    port (
        i_clk   : in std_logic;
        i_rst   : in std_logic;
        i_start : in std_logic;
        i_add   : in std_logic_vector(15 downto 0);
        i_k     : in std_logic_vector(9 downto 0);

        o_done  : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

Specifically:

- **i_clk** is the input CLOCK signal generated by the Test Bench;
- **i_rst** is the RESET signal that initialises the machine, ready to receive the first START signal;
- **i_start** is the START signal generated by the Test Bench;
- **i_add** is the ADD signal (vector) generated by the Test Bench, representing the address at which the sequence to be processed begins;
- **i_k** is the W signal (vector) generated by the Test Bench, representing the length of the sequence;
- **o_done** is the DONE output signal indicating the end of processing;
- **o_mem_addr** is the output signal (vector) that sends the address to memory;
- **i_mem_data** is the signal (vector) arriving from memory and containing the data following a read request;
- **o_mem_data** is the signal (vector) sent to memory containing the data to be written;
- **o_mem_we** is the WRITE ENABLE signal that must be sent to memory (=1) in order to write to it. To read from memory, it must be 0;
- **o_mem_en** is the ENABLE signal that must be sent to the memory in order to communicate (both read and write);

2 Architecture

2.1 Design Choices

To implement the required component, the approach was to divide the design into four main modules, each of which handles a different aspect of the hardware device. Each module interfaces with the finite-state machine we designed, which is responsible for combining and managing the signals to produce the expected output. A detailed description of each component follows.

2.2 DATA PATH

The first module we will analyse is the Data Path. It consists of two registers (reg1 and reg2) and a two-input multiplexer, and is responsible for managing the data read from memory. All registers are initialised to 0 prior to the first start signal.

Specifically, the word read (the i_data signal connected to i_mem_data from memory) is stored in register reg1 and compared with 0. The resulting signal (r1_eq0) will be used to allow the FSM to handle the different cases (data equal to zero or not equal to zero). In the reg2 register, on the other hand, the value preceding the input data is stored and also compared with zero. This register allows the data preceding the read data to be written to memory in the event that the latter is zero. The multiplexer, in fact, handles precisely this aspect: it selects which data to write to the output (o_data) and, depending on the processing, allows the contents of reg1 to be written rather than those of reg2.

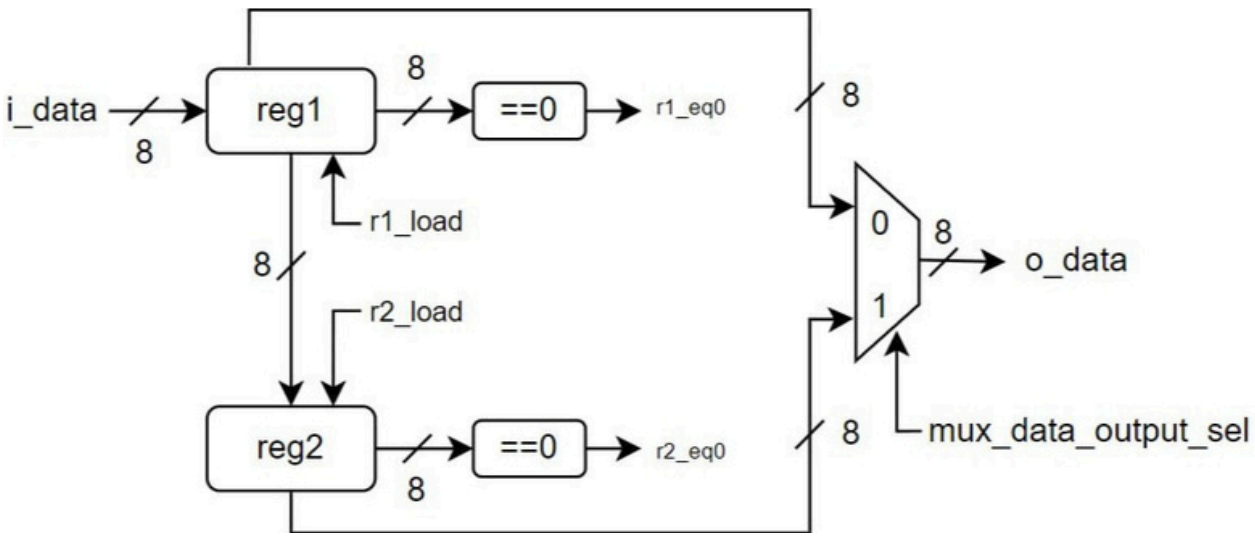

Figure 2: Data Path

2.3 CREDIBILITY_COUNTER

The second module we will examine is the Credibility Counter. This module is responsible for calculating the correct credibility value to be written to the output. It consists of a register (reg3), a four-input multiplexer and a subtractor (which subtracts one). The reg3 register stores the credibility value to be written to the output (o_data3) and, if necessary, its value is decremented by one via the subtractor. The multiplexer, on the other hand, allows the register's content to be set to 31 (the default credibility value) or to store the previous decremented value. Furthermore, it allows the credibility value to be maintained at 0 in the event that, following successive decrements, this value has been reached. Since the output of the Credibility Counter is the same as that of the Data Path (o_mem_data), we have added another two-input multiplexer (controlled by the o_mux_sel signal) which allows us to choose, based on the FSM's processing, whether to connect the signal from the Data or the one from the Credibility to the output.

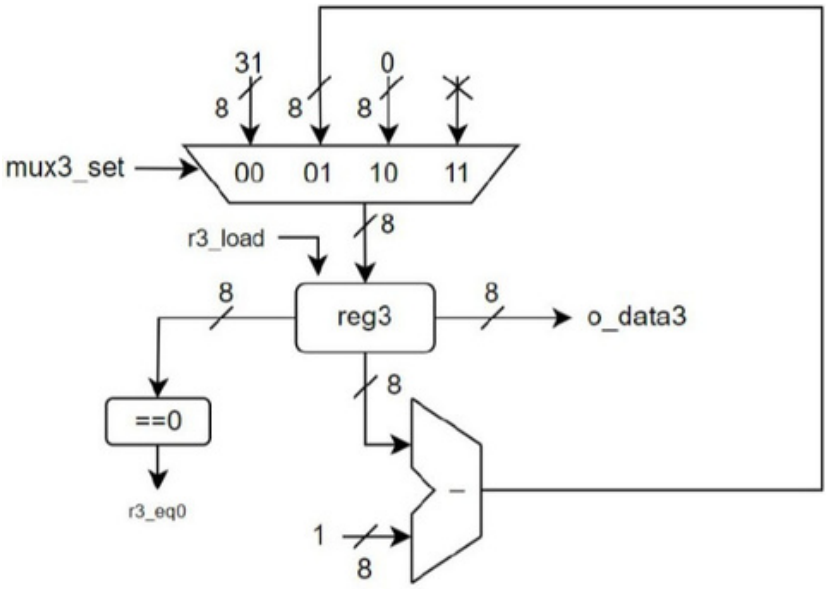

Figure 3: Credibility Counter

2.4 ADD_COUNTER

The Add Counter, on the other hand, calculates the correct memory address into which to write the data or the credibility. This module consists of a register (reg5), a two-input multiplexer and an adder. The input signal to the multiplexer is i_add (an input signal indicating the memory address from which to start reading); this multiplexer allows the value to be stored in reg5 (the register containing the memory address of the data or credibility) to be selected; this will be the starting address (i_add) for the first read and subsequently, as processing proceeds, it will be incremented by one each time (thus allowing access all addresses in the sequence). The output of the reg5 register will then be connected to the o_mem_addr output of the component.

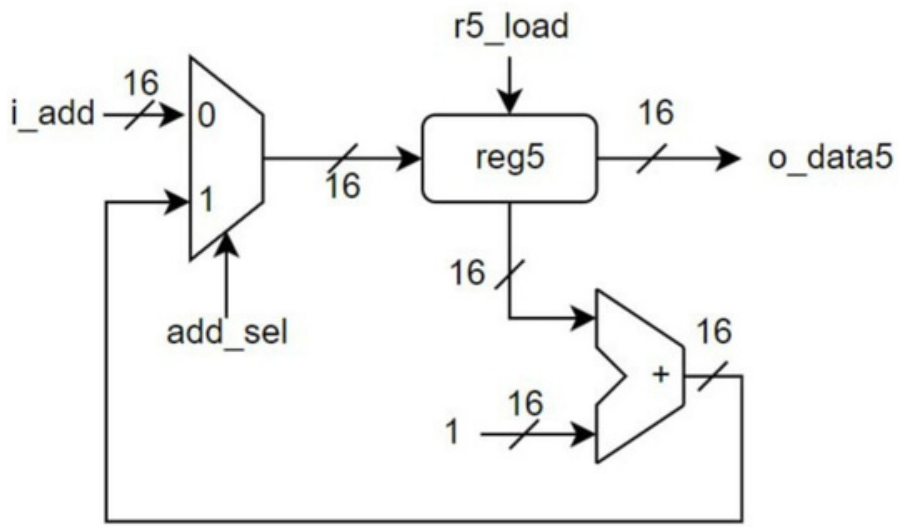

Figure 4: Add Counter

2.5 K_COUNTER

The final module we will examine is the K Counter, which is responsible for keeping track of the length of the sequence to be analysed. It consists of a register (reg4), a two-input multiplexer and a subtractor (which subtracts one). The input signal i_k indicates the length of the sequence and is stored in the register; the multiplexer allows the value of the signal, decremented by one, to be stored in reg4 each time a word is processed. When the content of reg4 reaches zero, the entire sequence has been processed and the computation can end.

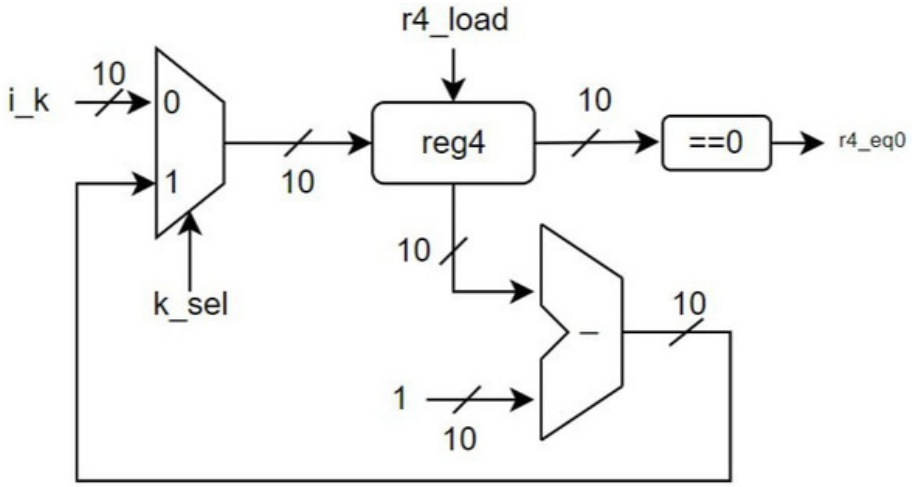

Figure 5: K Counter

2.6 FINITE STATE MACHINE (FSM)

The designed FSM handles the algorithmic partof the project and is responsible for processing and coordinating all the output signals from the various modules in order to produce the expected output. It consists of fifteen states, the behaviour of which we analyse below.

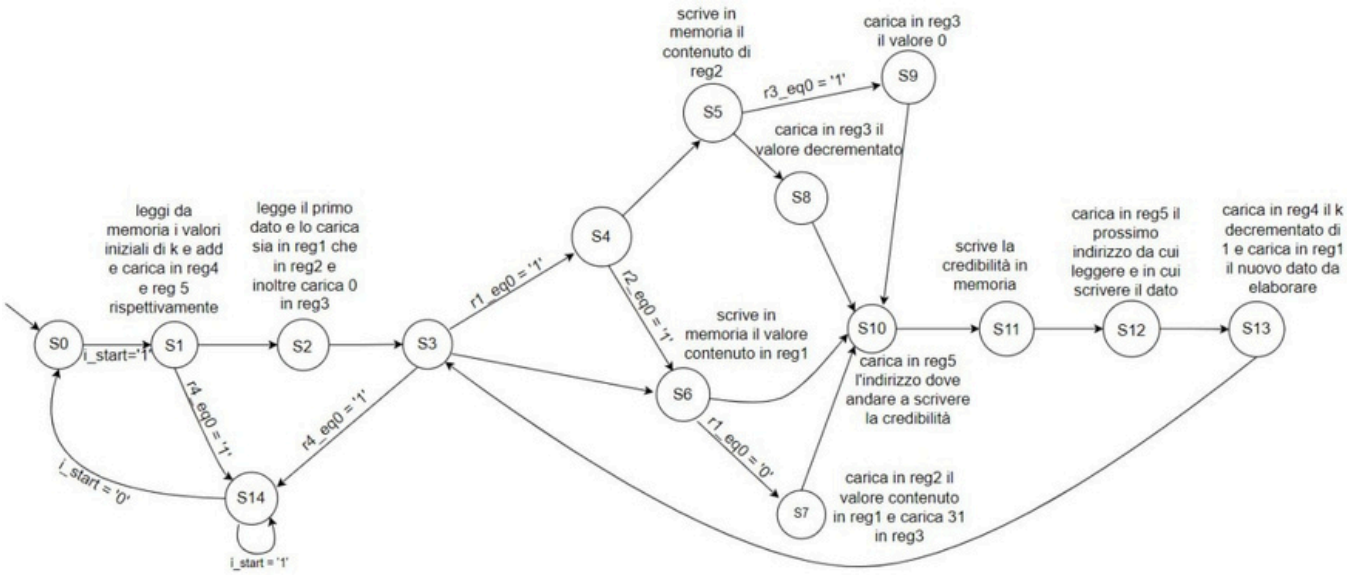

Figure 6: Finite-state machine

S0: Initial state. When the START signal is set to 1, the system transitions to the next state.

S1: o_mem_en = 1 allows me to read from memory; the sequence length is loaded into register reg4 (r4_load = 1 and mux_4 = 0), while the memory address from which processing is to start is stored in reg5 (r5_load = 1 and mux_5 = 0). If the length of the loaded sequence is 0 (r4_eq.0 = 1), the system moves directly to state S14.

S2: in this state of the FSM, the registers handling data and credibility are initialised. r1_load = 1 allows the first data to be read to be stored in reg1. Furthermore, by setting r2_load to 1, we ensure that the data read, which is also present in reg1, is stored in reg2, so that a subsequent processing operation does not find data belonging to the previous sequence in the second register (this is because, by specification, a RESET signal is not required before a new START signal). r3_load = 1 and mux_3 = 10 allow the reg3 register to be initialised to zero.

S3: This is a transition state. The contents of register reg1 (r1_eq.0) are evaluated, and depending on the result, the process moves on to the subsequent states.

S4: This is a transition state like S3. In this case, however, the contents of register reg2 (r2_eq.0) are evaluated and, depending on the result, the process moves on to the subsequent states.

S5: If the contents of reg1 are zero (r1_eq.0 = 1) but those of reg2 are not (the value prior to the one read is non-zero), then by setting both o_mem_we and o_mem_en to 1, data can be written to memory; and by setting mux_data_sel to 1 (the signal that controls the

(output multiplexer from the data path), the value contained in reg2 (previous data) will be written. Furthermore, by setting o_mux_sel = 0, I ensure that the signal from the Data Path is assigned to the o_mem_data output, rather than the one from Credibility.

S6: If, in addition to reg1, the content of reg2 is also zero (r2_eq.0 = 1) (a case where I have a sequence of zeros at the input from the start), then by setting the o_mem_we and o_mem_en signals to one, I enable writing to memory, to which the value contained in reg1 will be written (mux_data_sel = 0 and o_mux_sel = 0 to select the output value from the data path). This state, starting from the first non-zero data, will only be reached from S3 onwards; in fact, if the content of reg1 is non-zero, the value it contains (the data just read) will automatically be written to memory. Subsequently, from S6, state S10 can be reached or, if the contents of reg1 are non-zero, state S7.

S7: If the data read and contained in r1 is non-zero (I am considering this case where we have automatically reached S6 without passing through S4), then I load the value contained in reg1 into reg2 (r2_load = 1) and load the value 31 into register reg3 (credibility) (r3_load = 1 and mux_3 = 00).

S8: State reached automatically from S5. In this state, the decremented value is loaded into the credibility register (r3_load = 1 and mux_3 = 01).

S3: State reached from S5 if credibility has reached the value 0 (r3_eq.0 = 1). In this case, credibility is maintained at 0 (it does not decrease below this value) (r3_load = 1 and mux_3 = 10).

S10: State reached automatically from S6, S7, S9 and S8. In this state, the memory address from which to read the data or credibility is incremented (r5_load = 1 and mux_5 = 1). The system automatically transitions to state S11.

S11: In this state, after processing the word to be read according to the various cases and calculating the credibility, setting o_mem_we, o_mem_en and o_mux_sel to 1 enables writing to memory and the credibility value is written.

S12: State reached automatically from S11. In this state, the memory address is incremented by one again, so as to access the next data to be read (r5_load = 1 and mux_5 = 1).

S13: In this state, the r1 register is loaded again with the data to be processed (r1_load = 1) and the sequence length is decremented by one (r4_load = 1 and mux_4 = 1). From here, the process returns directly to S3 to process the new word.

S14: If the value of r4 is zero (r4_eq.0 = 1) (the sequence length is zero and processing has therefore finished), the output o_done is set to 1 and remains in this state until i_start returns to zero. When this happens, the process returns to S0 and restarts.

3 EXPERIMENTAL RESULTS

3.1 Bench Tests

To verify the correct operation of the synthesised component, we carried out various testing activities, running both the tests provided by the lecturers and some tests created specifically to verify boundary cases of interest to us and to ensure that the code functioned in all possible scenarios. The graphs below show the behaviour of the component in these boundary cases.

1. Case K = 0 (empty sequence)

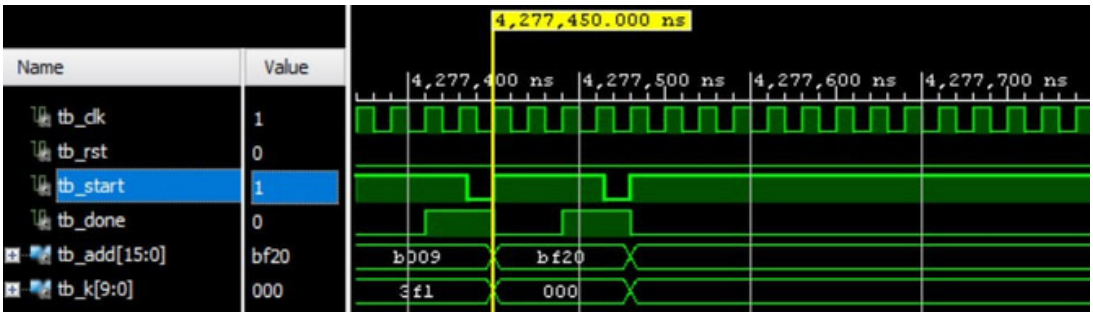

Figure 7: Test 1

In this case, the sequence to be analysed is of length 0 and the DONE signal is immediately set to 1, terminating the computation, as specified.

2. Multi-sequence and sequence consisting solely of 0s

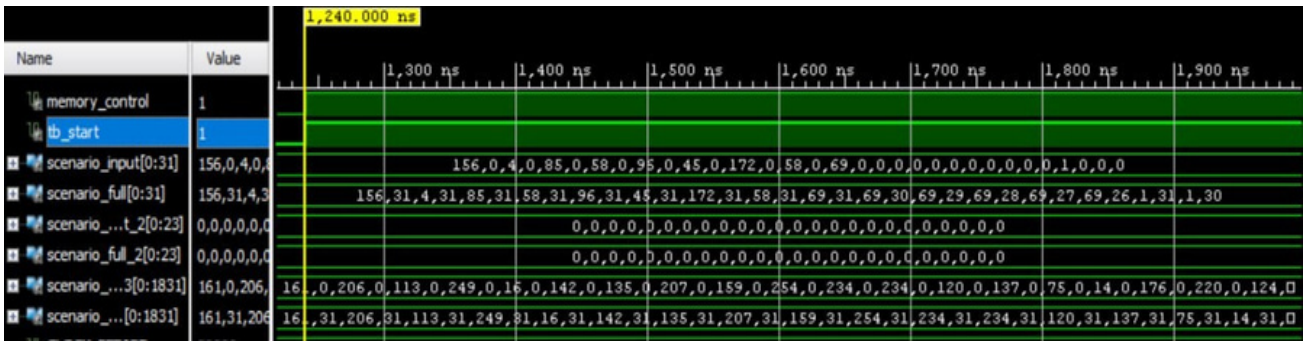

Figure 8: Test 2

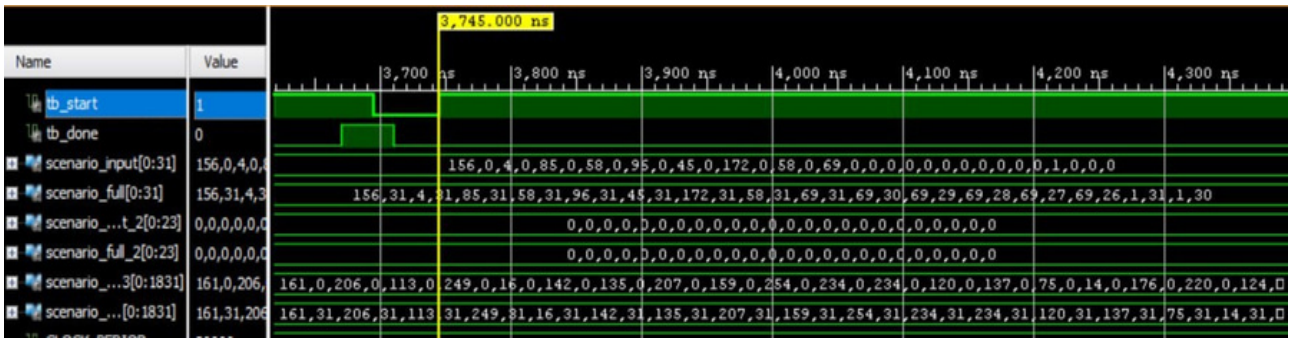

Figure 9: Test 3

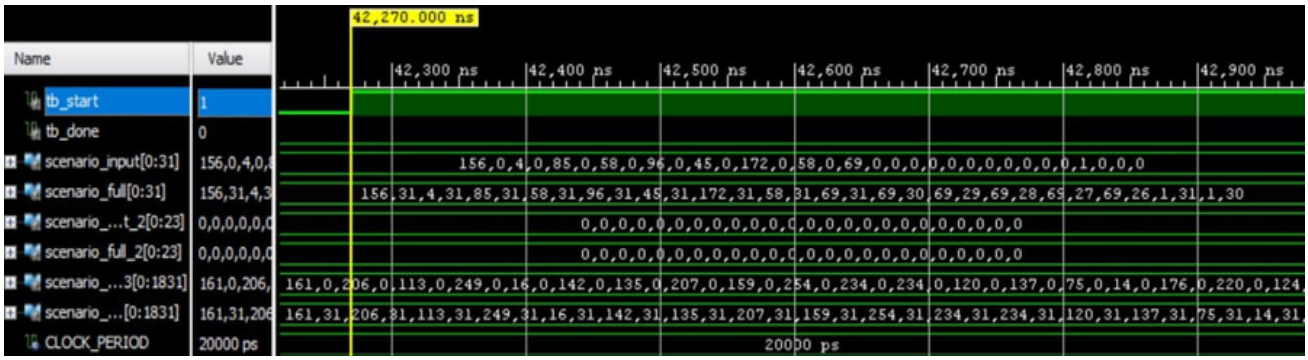

Figure 10: Test 4

With this test, we verified the correctness of the processing in cases where there are multiple consecutive sequences at the input. We found that the component correctly resets the registers when moving from one string to the next; furthermore, as shown in the image, it also calculates correctly when the input consists of a sequence of zeros only. In this latter situation, the component leaves the memory data unchanged without specifying the credibility, as indicated in the specification.

3. Reset

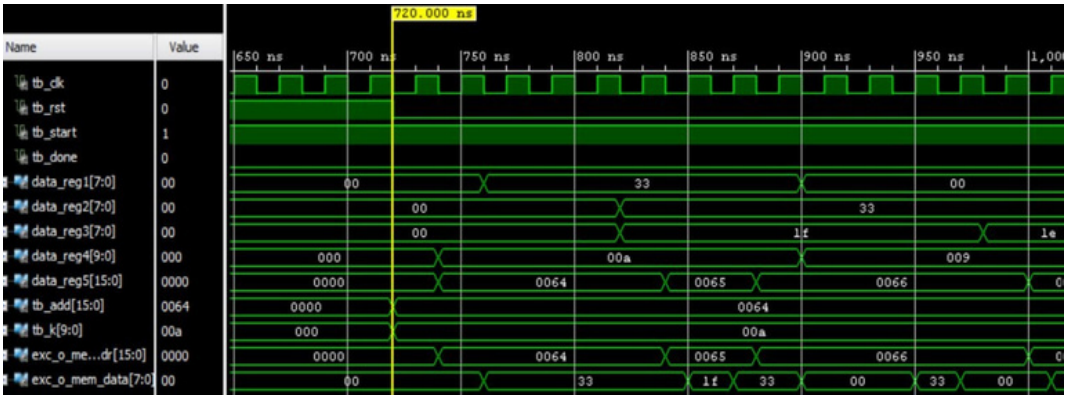

Figure 11: Test 5

In these cases, we verified that the component functioned correctly even when a RESET signal was applied. Specifically, *Figure 11* tests the case where both the RESET and START signals are set to 1. As can be seen

in the image, the registers are updated correctly only when the RESET signal is returned to 0.

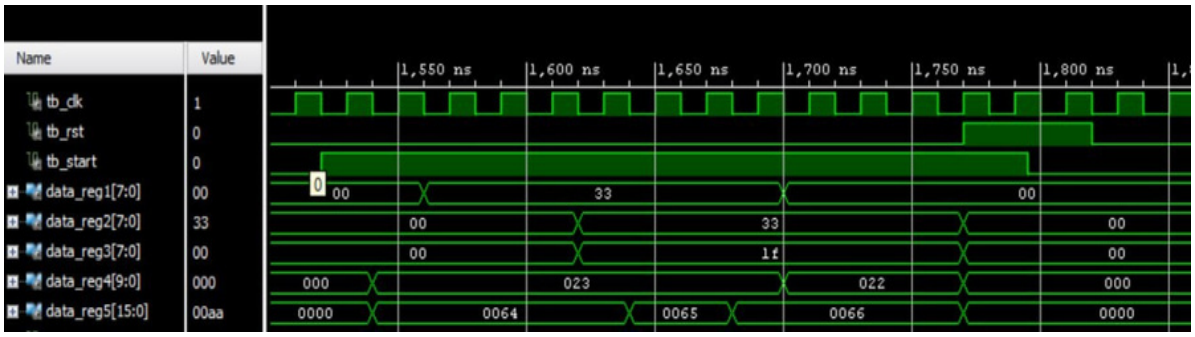

Figure 12: Test 6

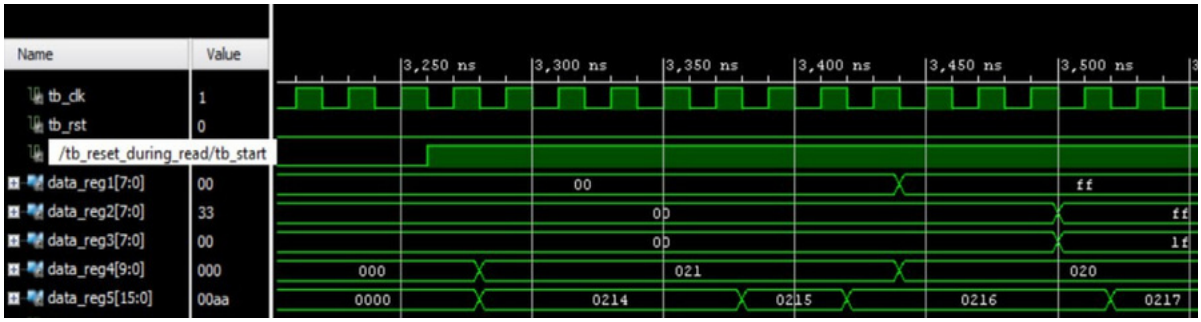

Figure 13: Test 7

In *Figures 12* and *13*, however, the RESET is set to 1 during the processing of a sequence and, as can be seen, as soon as the signal to reset the component is applied, all registers are reset to 0 (*Figure 12*) and the component resumes processing correctly with a new START signal (*Figure 13*)

4. Credibility reduced to zero (and beyond)

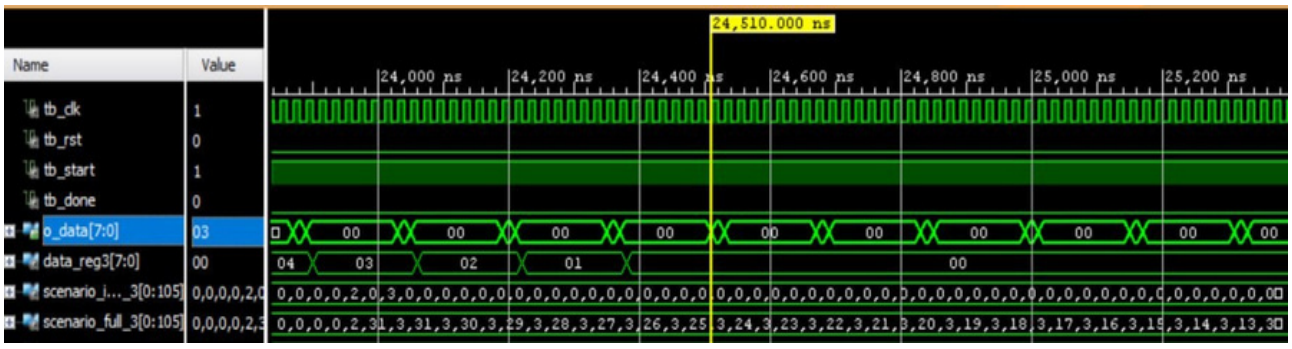

Figure 14: Test 8

With this test, we verified the correct functionality of the component even in the event that a series of 0s at the input brings the credibility down to its limit value
(0). In *Figure 14*, it can be clearly seen that, once the content of the credibility register reaches zero (content of data_reg3), it remains at that value without being further decremented.

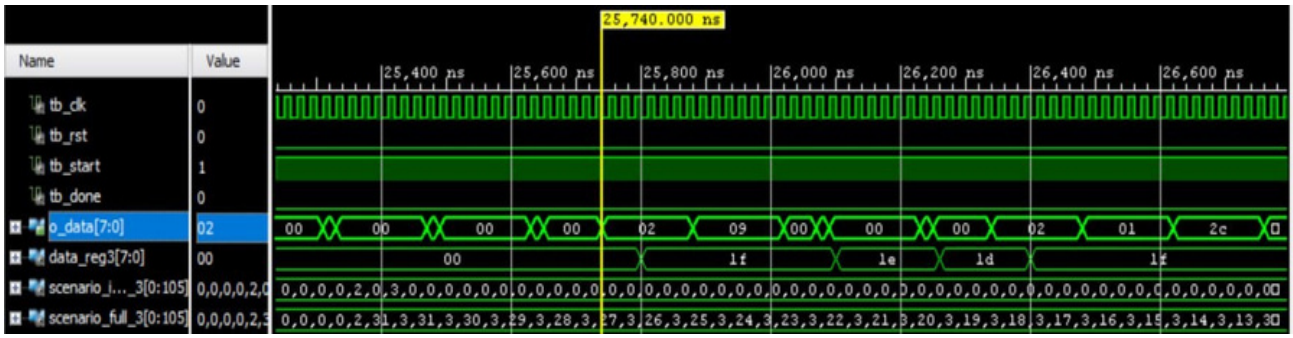

Figure 15: Test 9

In *Figure 15,* on the other hand, we can observe the case where the input data once again assumes a valid value (other than zero) and the credibility correctly reverts to its default value (31).

3.2 Summary Report

1. Utilisation Report

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	94	0	134600	0.07
LUT as Logic	94	0	134600	0.07
LUT as Memory	0	0	46200	0.00
Slice Registers	64	0	269200	0.02
Register as Flip Flop	64	0	269200	0.02
Register as Latch	0	0	269200	0.00
F7 Muxes	3	0	67300	<0.01
F8 Muxes	0	0	33650	0.00

The Utilisation report shows that the project has no latches, confirming the soundness of the design choices and the operating logic.

2. Timing Report

```
Timing Report

Slack (MET) :            1.993ns  (required time - arrival time)
  Source:                FineStateMachine/FSM_sequential_cur_state_reg[3]/C
                           (rising edge-triggered cell FDCE clocked by clock  {rise@0.000ns fall@5.000ns period=20.000ns})
```

The timing report confirms that the design meets the timing requirements specified in the specifications; although it is close to reaching the maximum limit, the component is functioning correctly.

4 CONCLUSIONS

The implementation of this project differed from others we have undertaken; it forced us to set aside the sequential way of thinking we are accustomed to in favour of a more complex form of logical reasoning, one that is also closer to the actual physical representation of the hardware components.
During development, we encountered a number of difficulties, ranging from understanding the tool we were using to debugging our code by analysing the component's signal waveforms; this proved to be not only the most challenging aspect but also the most useful for gaining a deeper understanding of the subject.
Although we are aware that the state machine could have been simplified even with a design that did not involve a division into sub-modules, the implementation we decided to adopt was chosen to make the component as easy as possible for an external observer to understand and, in our view, also more modular.
The module, designed in this way, behaves correctly in accordance with the assigned specification and successfully passes all the tests specified in the behavioural and post-synthesis simulations.